

QMWS Spring 2026: Introduction to Python for Data Analysis

Day One: Foundations

Gavin Klorfine & Deborah Laze

Section 1

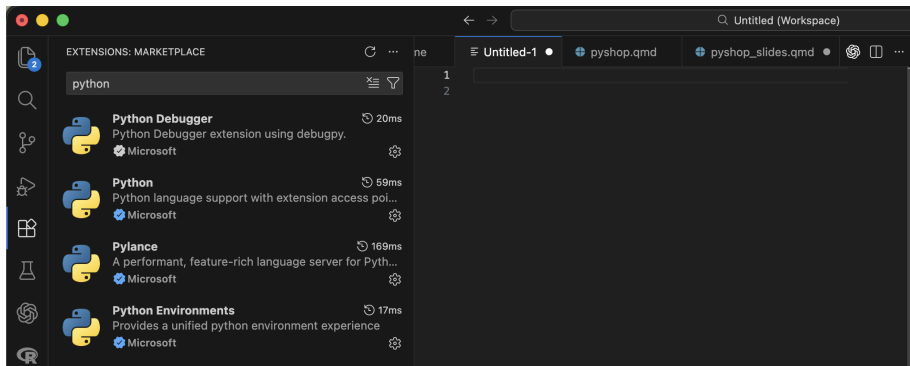
Installation

Visual Studio (VS) Code

- Installation link: <https://code.visualstudio.com/download>
- Integrated Development Environment (IDE)
- Can use it to code in many different languages
- Commonly used in both academia and industry

Visual Studio (VS) Code

- Once installed, go to the “Extensions” tab and search “Python”
- Install the below extensions:



- A tool for configuring Python **environments**
 - More often than not, you need to install **packages** that allow you to code more easily/efficiently
 - Allows for version control
 - Issues can arise if environments are not used, for example:
 - If you install two packages that conflict with one another
 - If you install a package that requires a specific version of Python
 - Anaconda allows these issues to be isolated to a specific environment, rather than affecting your whole Python installation
 - Especially useful if you have more than one project going on

- Installation link: <https://www.anaconda.com/download>
- The simplest way to use Anaconda is via the **Navigator**
- To install:
 - 1 Make an account
 - 2 Install the full distribution via the “Graphical Installer”

- Installation link: <https://www.anaconda.com/download>
- On Mac:



Choose Your Download

Windows

Mac

Linux

Anaconda Distribution

Complete package with 8,000+ libraries, Jupyter, JupyterLab, and Spyder IDE. Everything you need for data science.

↓ 64-Bit (Apple silicon)
Graphical Installer

↓ 64-Bit (Apple silicon)
Command Line Installer

Miniconda

Minimal installer with just Python, Conda, and essential dependencies. Install only what you need.

↓ 64-Bit (Apple silicon)
Graphical Installer

↓ 64-Bit (Apple silicon)
Command Line Installer

↓ 64-Bit (Intel chip) Graphical
Installer

- Installation link: <https://www.anaconda.com/download>
- On Windows:



Choose Your Download

Windows

Mac

Linux

Anaconda Distribution

Complete package with 8,000+ libraries, Jupyter, JupyterLab, and Spyder IDE. Everything you need for data science.

↓ [Windows 64-Bit Graphical Installer](#)

Miniconda

Minimal installer with just Python, Conda, and essential dependencies. Install only what you need.

↓ [Windows 64-Bit Graphical Installer](#)

Section 2

Setting Up a Conda Environment

Section 3

The Basics

- In programming, a variable is a space in computer memory where values may be stored

Defining and Printing Variables

```
x = 67
```

```
print(x)
```

```
67
```

Defining and Printing Variables

```
a = 1
```

```
b = 2
```

```
print(a + b)      # * for multiplication, / for division
```

```
3
```

Note

- Use # to make a **comment**

Variable Types

- ❶ `int` (integer; “whole number”)
 - less memory
 - 1 is an `int`
- ❷ `float` (real; “decimal number”)
 - 1.2 is a `float`
 - 1.0 is a `float`
- ❸ `str` (string; characters/words with no numerical meaning)
 - “Hello world!” is a string
 - “416-676-6767” is also a string
- ❹ `bool` (boolean; can be `True` or `False`; more on this later)
- ❺ `None`
 - A special type of variable that essentially means “nothing meaningful is stored”
 - Useful as a placeholder

Variable Types

Checking Type

```
x = 416
print(x, type(x))    # type() to check type

416 <class 'int'>
```

Converting Type

```
x = 416
x = str(x)           # str() to convert to string
print(x, type(x))

416 <class 'str'>
```

- Can also use `int()` and `float()` to convert to respective type

A Brief Note

- `print()`, `type()`, `str()`, etc. are all examples of **functions**
 - These are essentially pre-made chunks of code for you to use
 - More on this later!

What will be outputted by the below code?

```
x = "Variable"  
  
print(type(x))  
print(x)
```

Section 4

More Advanced Variable Types

- Multiple values can be stored in one variable. A list is one way to do this:

Defining Lists

```
x = [0, 1, 2, 3, 4]      # Define a list; square brackets  
print(type(x))
```

```
<class 'list'>
```

Lists: Indexing

- To extract one or more values from a list, you **index** the list
- Python uses **zero-based indexing**
 - This means that the first element of a list is index 0, the second element is index 1, ...
 - Element x is index $x - 1$

Indexing Lists

```
x = [0, 1, 2, 3, 4]
```

```
print(x[0])      # Index first element of a list
```

```
print(x[-1])     # Index last element
```

```
0
```

```
4
```

Lists: Indexing

- You can also take more than one element, or a **slice**, of the list

Slicing Lists

```
x = [0, 1, 2, 3, 4]
```

```
print(x[0:2])    # first and second (but not third) elements
```

```
[0, 1]
```

! Important

- `[inclusive:exclusive]`
 - The index on the *left-hand* side of the slice is *included*, whereas the index on the *right-hand* side of the slice is *excluded*

Slicing Lists, continued

```
x = [0, 1, 2, 3, 4]
```

```
print(x[0:3])    # first three elements
```

```
print(x[0:1])    # a (silly) way to index just the first element
```

```
[0, 1, 2]
```

```
[0]
```

Lists: Length

- To find the length of a list:

len() function

```
x = [0, 1, 2, 3, 4]
```

```
print(len(x))    # The length of list x
```

```
# Another way to find the last element (not so silly!)
```

```
print(x[len(x) - 1])
```

```
5
```

```
4
```

Lists: Methods

- Lists have certain functions that can be applied to them, called **methods**
 - Methods are more general than this (e.g., string methods; more on these in a little bit!)

```
.append()
```

```
x = [0,1,2,3,4]
```

```
x.append(5)
```

```
print(x)
```

```
[0, 1, 2, 3, 4, 5]
```

- Adds an entry to a list

`.extend()`

```
x = [0,1,2,3,4]
```

```
x.extend([6,7,8,9])
```

```
print(x)
```

```
[0, 1, 2, 3, 4, 6, 7, 8, 9]
```

- Extend a list by “glueing” it to another list

Difference Between `.append()` and `.extend()`

```
x = [0,1,2,4]
```

```
y = [5,6,7,8]
```

```
x.append(y)
```

```
print(x)
```

```
x = [0,1,2,4]    # Redefine x
```

```
x.extend(y)
```

```
print(x)
```

```
[0, 1, 2, 4, [5, 6, 7, 8]]
```

```
[0, 1, 2, 4, 5, 6, 7, 8]
```

Lists: Changing Values

Changing a Value in a List

```
x = [0, 1, 2, 3, 4]
```

```
x[0] = 9
```

```
print(x)
```

```
[9, 1, 2, 3, 4]
```

Tuples (Vriefly)

- Like lists, tuples contain multiple values
- Unlike lists, tuples are **immutable**; they cannot be changed once created.
 - Lists are *mutable*

Defining a Tuple

```
x = ("a", "tuple")  # Round brackets
```

```
print(x[0], x[1])
```

```
print(type(x))
```

```
a tuple
```

```
<class 'tuple'>
```

Tuples (Briefly)

Changing Values... ERROR!

```
x = ("a", "tuple")
```

```
x[0] = "Not a"
```

TypeError: 'tuple' object does not support item assignment

TypeError

Traceback (most recent call last):

Cell In[16], line 3

```
1 x = ("a", "tuple")
```

```
----> 3 x[0] = "Not a"
```

TypeError: 'tuple' object does not support item assignment

Tuples (Briefly)

- Tuples have special properties, one of which is below:

Unpacking Tuples

```
x, y = (6, 7)
```

```
print(x)
```

```
print(y)
```

```
6
```

```
7
```

Section 5

Strings

- Strings are a lot like lists in that they are **iterable**
 - For now, think of **iterable** items (lists, strings) as a sequence of values